"Empowerment through Technological Excellence" GENBA
SOPANRAO MOZE COLLEGE OF ENGINEERING
(Recognized by AICTE, New Delhi; Approved by Govt. of Maharashtra;Affiliated to Pune University)
Balewadi, Pune – 411045, Ph: 020-27390500
Website: www.gsmozecoe.co.in Email: gsmoze@yahoo.co.in

Computer Engineering

Lab Manual

Final Year Computer Engineering

Class: Final Year

Semester-II

Laboratory Practice -V

Academic year 2025-26

410255: Laboratory Practice V

Teaching Scheme	Credit	Examination Scheme
PR: 02 Hours/Week	01	TW : 50 Marks PR : 50 Marks

Guidelines for Instructor's Manual

Laboratory Practice V is for practical hands on for core courses High Performance Computing and Data Learning. The instructor‘s manual is to be developed as a hands-on resource and as ready reference. The instructor's manual need to include prologue (about University/program/ institute/ department/foreword/ preface etc), University syllabus, conduction and Assessment guidelines, topics under consideration-concept, objectives, outcomes, set of typical applications/assignments/ guidelines, references among others.

Guidelines for Student Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal may consists of prologue, Certificate, table of contents, and handwritten write-up of each assignment (Title, Objectives, Problem Statement, Outcomes, software and Hardware requirements, Date of Completion, Assessment grade/marks and assessor's sign, Theory- Concept in brief, Algorithm/Database design, test cases, conclusion/analysis). Program codes with sample output of allperformed assignments are to be submitted as softcopy.

Guidelines for Laboratory /Term Work Assessment

Continuous assessment of laboratory work is to be done based on overall performance and lab assignments performance of student. Each lab assignment assessment will assign grade/marks based on parameters with appropriate weightage. Suggested parameters for overall assessment as well as each lab assignment assessment include- timely completion, performance, innovation, efficient codes, punctuality and neatness reserving weightage for successful mini-project completion and related documentation.

Practical No.	Laboratory Assignments
High Performance Computing	
1	Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS.
2	Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.
3	Implement Min, Max, Sum and Average operations using Parallel Reduction.
4	Implement HPC application for AI/ML domain.
5	**Mini-Project** I. Mini Project: Evaluate performance enhancement of parallel Quicksort Algorithm using MPI II. Mini Project: Implement Huffman Encoding on GPU III. Mini Project: Implement Parallelization of Database Query optimization IV. Mini Project: Implement Non-Serial Polyadic Dynamic Programming with GPU Parallelization

Assignment No:- 1

Problem statement: Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS.

- # Aim

  Write a program to design and implement parallel Breadth First Search and Depth First Search algorithm

- # Prerequisites

  - Concept of existing sequential algorithms.
  - Concept of High Performance Computing.

- # Learning Objectives

  - To understand concept of Breadth First Search and Depth First Search based on sequential algorithm.
  - To understand concept of parallel algorithm.
  - To compare performance by varying number of processors used and also with sequential algorithm.

- # Learning Outcomes

After successfully completing this assignment, you should be able to

- Display result for parallel Breadth First Search and Depth First Search
- Analyze performance by varying number of processors used and also with sequential algorithm.
- Calculate speedup, efficiency, throughput

Concepts related Theory :- Breadth First Search(BFS)

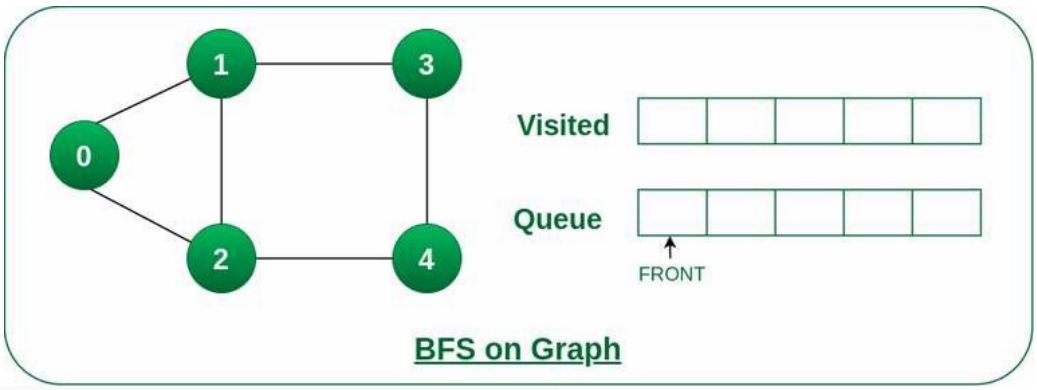
The breadth-first search (BFS) algorithm is used to search a tree or graph data structure for a node that meets a set of criteria. It starts at the tree's root or graph and searches/visits all nodes at the current depth level before moving on to the nodes at the next depth level.

Follow the below method to implement BFS traversal.

- Declare a queue and insert the starting vertex.
- Initialize a **visited** array and mark the starting vertex as visited.
- Follow the below process till the queue becomes empty:
  - Remove the first vertex of the queue.
  - Mark that vertex as visited.
  - Insert all the unvisited neighbors of the vertex into the queue.

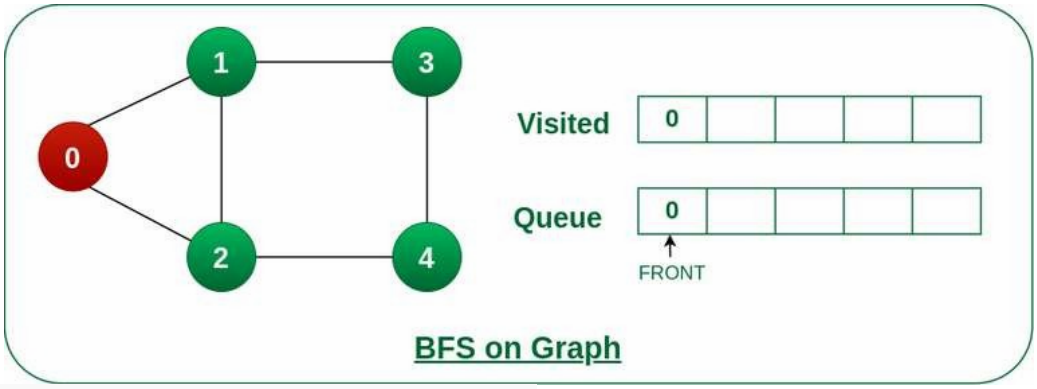
Illustration:

Step1: Initially queue and visited arrays are empty.



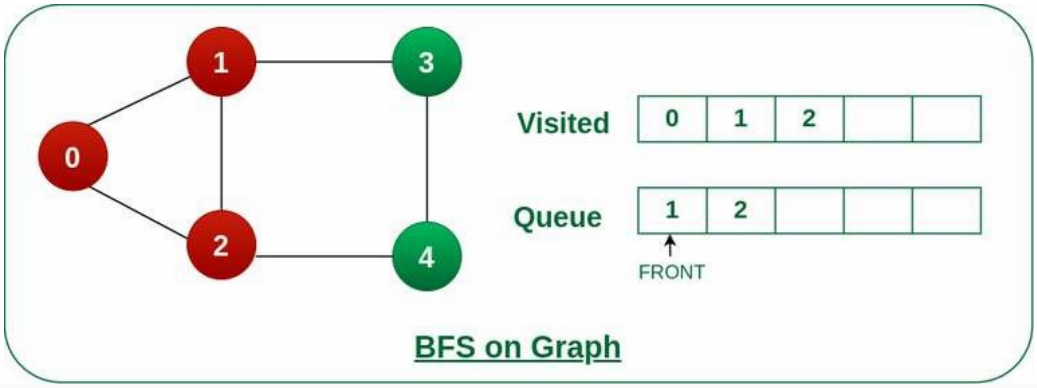
Queue and visited arrays are empty initially.

Step2: Push node0 into queue and mark it visited.



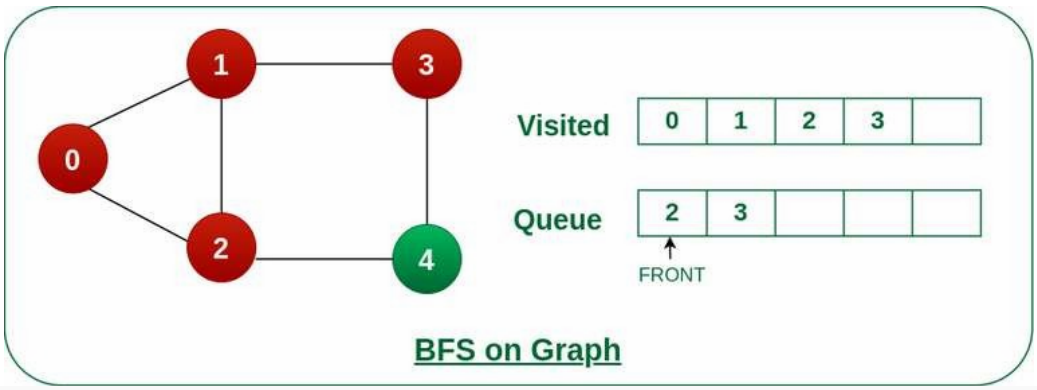
Push node0 into queue and mark it visited.

Step 3: Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.



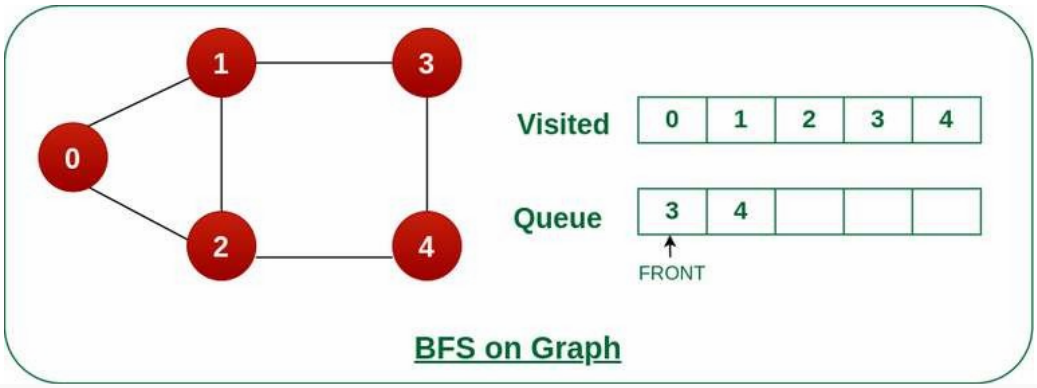
Remove node 0 from the front of queue and visited the unvisited neighbours and push into queue.

Step 4: Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



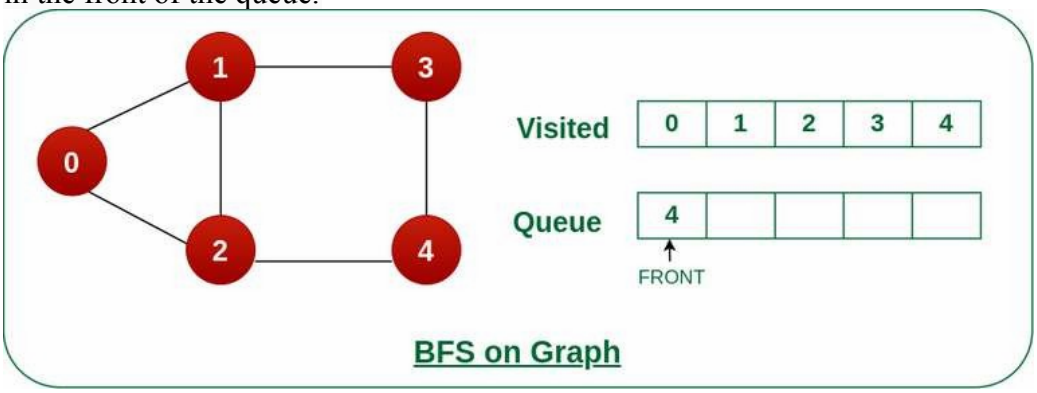
Remove node1 from the front of queue and visited the unvisited neighbours and push.

Step 5: Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



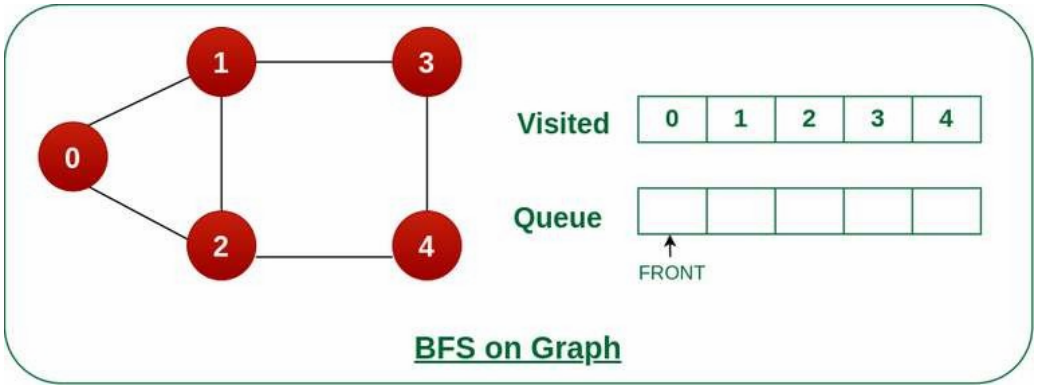
Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.

Step 6: Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.
As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.



Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.

Steps 7: Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.
As we can see that every neighbours of node 4 are visited, so move to the next node that is in the front of the queue.



Remove node 4 from the front of queue and visit the unvisited neighbours and push them

into queue. Now, Queue becomes empty, So, terminate these process of iteration.

Depth First Search Algorithm

A standard DFS implementation put search vertex of the graph into one of two categories:

1. Visited
2. Not Visited

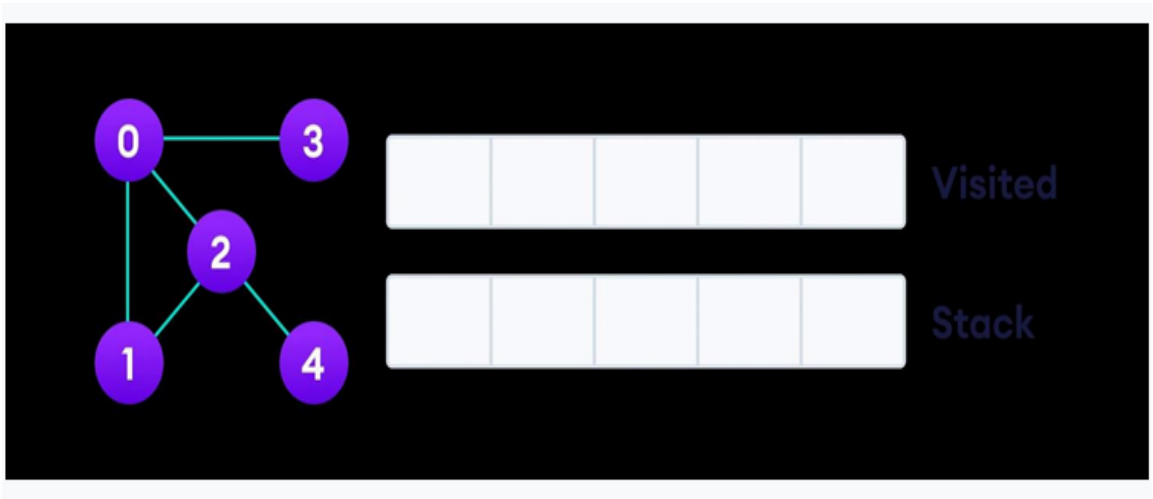
The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The DFS algorithm works as follows:

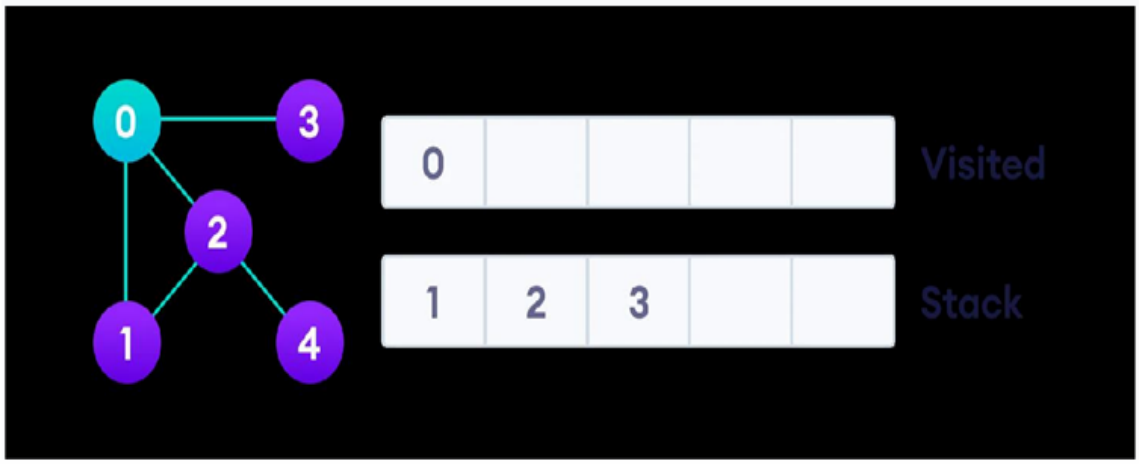
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating Step 2 & Step 3 until the stack is empty.

Depth First Search Example

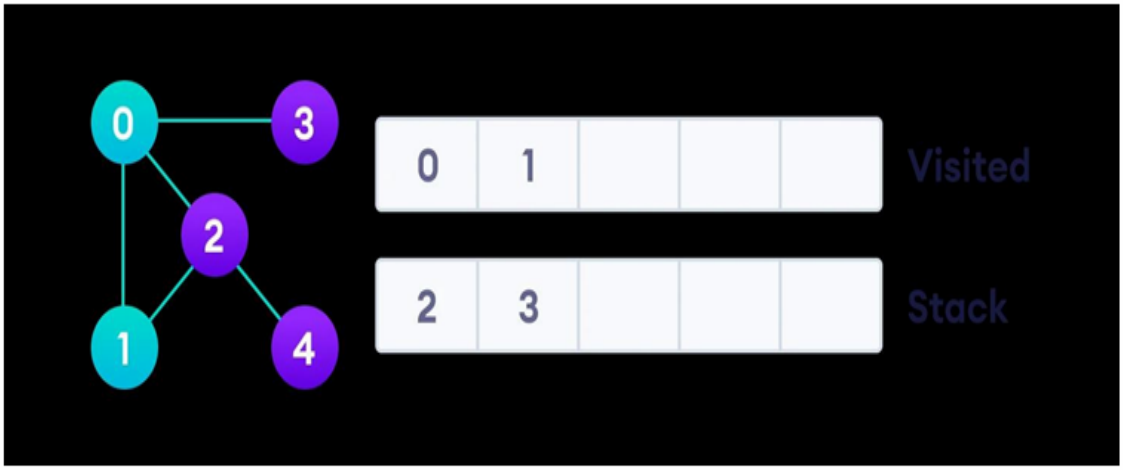
Let's see how the Depth First Search algorithm works with an example. We use an undirected graph with 5 vertices.



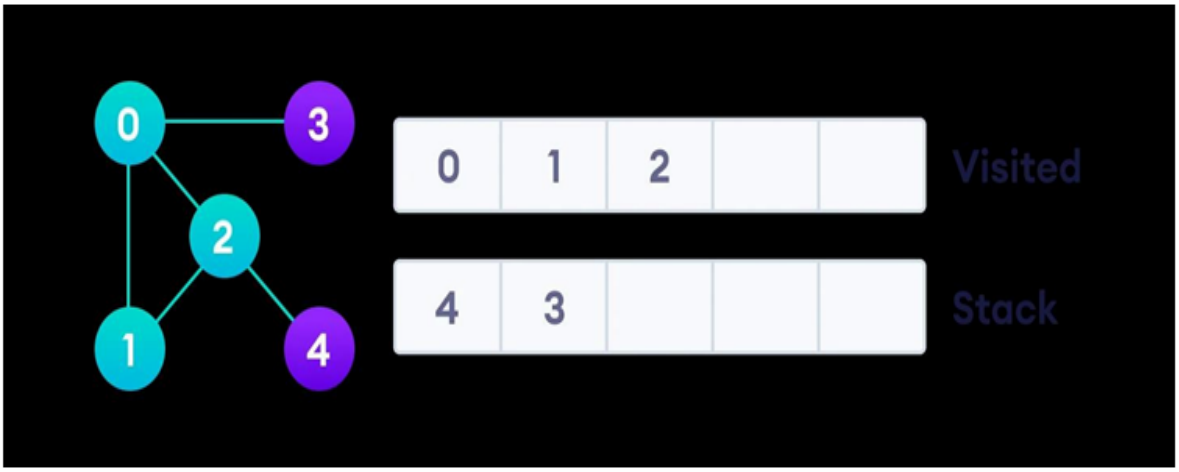
We start from vertex0, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.



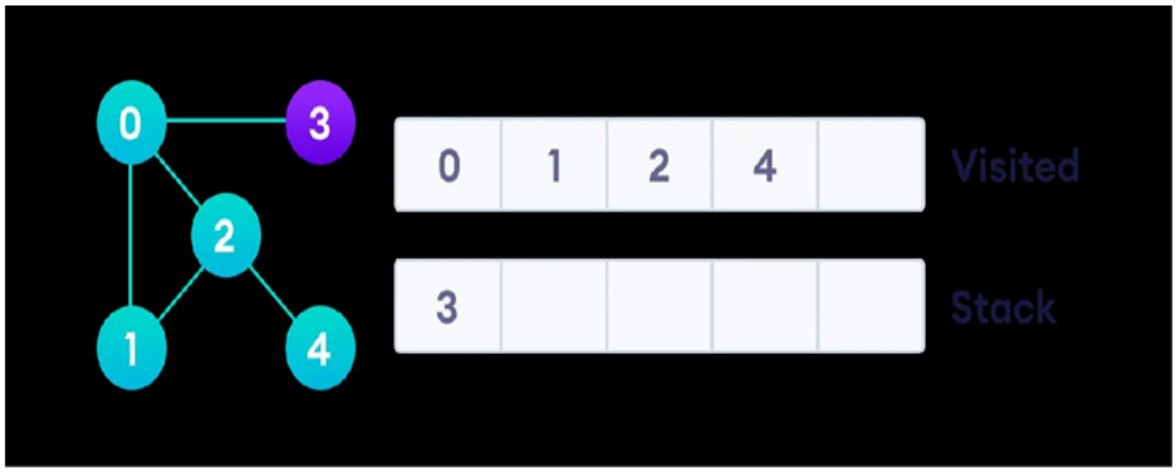
Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since0 has already been visited, we visit 2 instead.



Vertex 2 has an unvisited adjacent vertex in 4,so we add that to the top of the stack and visit it.

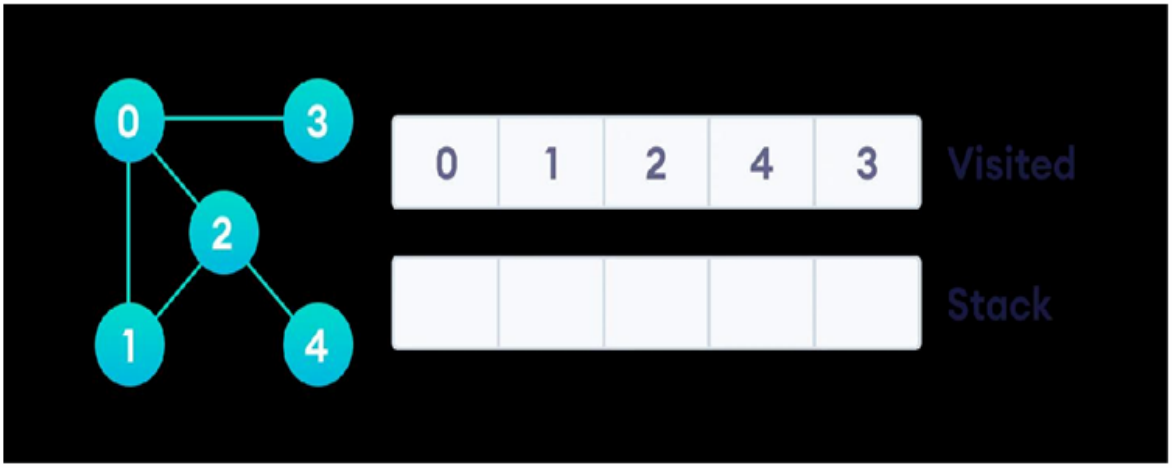


Vertex 2 has an unvisited adjacent vertex in 4,so we add that to the top of the stack and visit it.



Vertex 2 has an unvisited adjacent vertex in 4,so we add that to the top of the stack and visit it.

After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.



After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.

Parallel DFS and BFS: Extract parallelism from existing sequential algorithms and implement using OPENMP

Conclusion:

After successfully completing this assignment, student should be able to understandand implement parallel BFS and DFS in OpenMP.

Assignment No:- 2

Problem statement: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

- **Aim**
  Write a program to design and implement parallel Bubble Sort and Merge Sort algorithm.

- **Prerequisites**
- Concept of existing sequential algorithms.
- Concept of High Performance Computing.

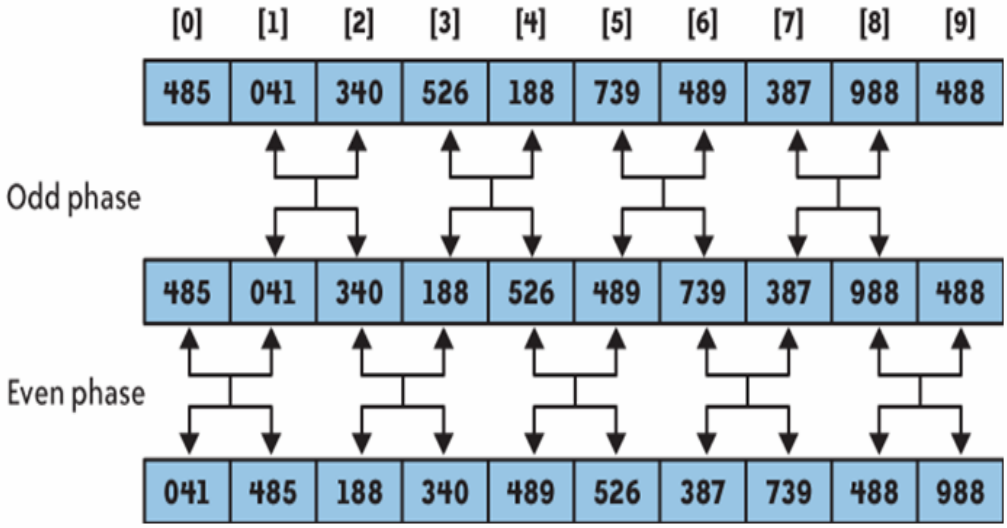
- **Learning Objectives**
- To understand concept of Bubble Sort and Merge Sort based on sequential algorithm.
- To understand concept of parallel algorithm.
- To compare performance by varying number of processors used and also with sequential algorithm.

- **Learning Outcomes**
  After successfully completing this assignment, you should be able to
- Display result for parallel Bubble Sort and Merge Sort.
- Analyze performance by varying number of processors used and also with sequential algorithm.
- Calculate speed up, efficiency, throughput

- **Concepts related Theory :-**

- **Bubble Sort Algorithm:-**
  Sequential Bubble Sort Algorithm:
  One of the straight-forward sorting methods
  –Cycles through the list
  –Compares consecutive elements and swaps them if necessary
  –Stops when no more out of order pair.
  _Slow & inefficient
  _Average performance is $O(n^2)$.

  Parallel Bubble Sort
  Compare all pairs in the list in parallel.
  When to stop?
  Shared flag, sorted, initialized to true at beginning of each iteration (2phases),if any processor perform swap, sorted = false



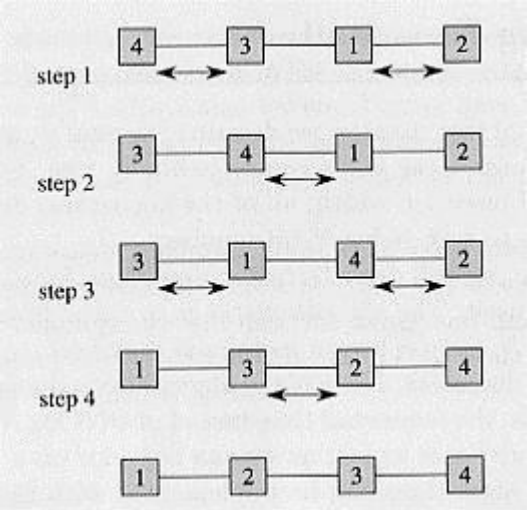
Parallel Bubble Sort Complexity

Sequential bubble sort, $O(n^2)$.
Parallel bubble sort ? (if we have unlimited #of processors)
Do n-1 comparisons for each iteration => O(n)

Parallel Bubble Sort Example:

How many steps does it take to sort the following sequence from least to greatest using the Parallel Bubble Sort ? How does the sequence look like after 2 cycles ?

•4,3,1,2



• Merge Sort Algorithm:-

Sequential Merge Sort:

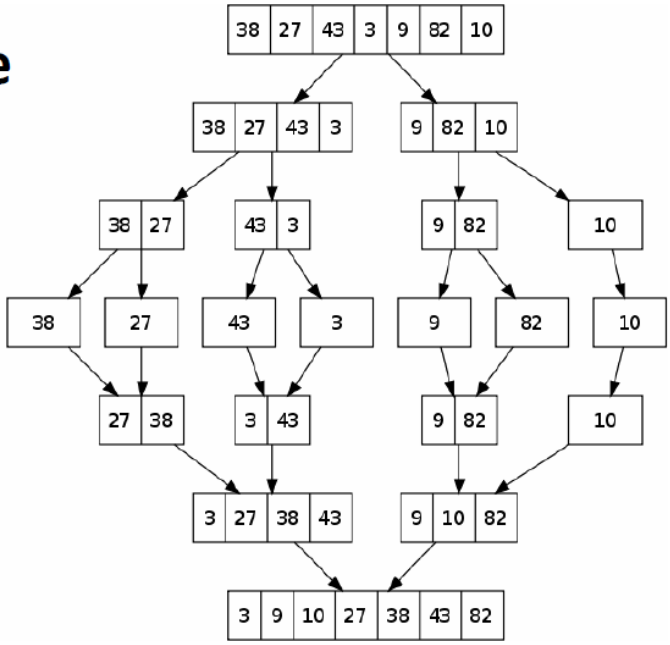
Divide and Conquer:

•Dividing problem into sub-problems
•Division usually done through recursion
•Solutions from sub-problems then combined to give solution to the original problem.

Collects sorted list onto one processor

Merges elements as they come together
•Simple tree structure
•Parallelism is limited when near the root

Example



Merge Sort Complexity:

Solve the recurrence
relation T(n) = O(nlogn)

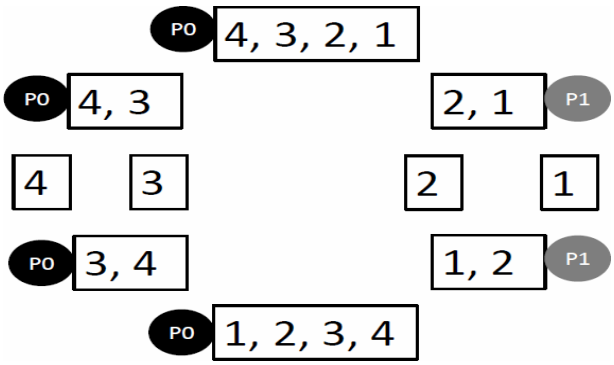
$$T(n) = \begin{cases} b & n = 1 \\ 2T\left(\frac{n}{2}\right) + bn & n > 1 \end{cases}$$

Parallel Merge Sort:

•Parallelize processing of sub-problems
•Max parallelization achieved with one processor per node (at each layer/height).

Parallel Merge Sort Example:

Perform Merge Sort on the following list of elements. Given 2 processors, P0&P1, which processor is responsible for which comparison?
•4,3,2,1



Parallel Merge Sort Complexity:

- Merge sort, O(n log n)
- Easy way to remember complexity, n(elements) x log n (tree depth)
- If we haven processors, O(log n)

Test Cases:

Students should write test cases depending on their input and test the program on large input data

Conclusion:

After successfully completing this assignment, student should be able to understand and implement parallel bubble sort and merge sort in OpenMP.

Assignment No:- 3

- **Aim**
  Write a program to implement Min, Max, Sum and Average using Parallel Reduction.

Objectives:

- To understand the concept of parallel reduction
- To apply reduction operations on arrays
- To implement reduction using parallel programming (OpenMP)

What is Parallel Reduction?

Parallel Reduction is a technique used in parallel computing to reduce a large dataset into a single value by performing operations in parallel. It divides the data into smaller parts, processes them simultaneously, and combines the results.

Reduction Operations

1 **Sum Reduction**
Combines elements using addition.

2 **Min Reduction**
Finds smallest element.

3 **Max Reduction**
Finds largest element.

4 **Average**
Calculated using:

Average= Sum / Total Elements

Algorithm:

1. Start
2. Read array elements
3. Divide array into parts
4. Perform pairwise reduction
5. Repeat until one value remains
6. Compute average using sum
7. Display results

Result:

Thus, Min, Max, Sum and Average were successfully computed using Parallel Reduction.

Assignment No:- 4

Problem Statement:

Implement HPC application for AI/ML domain

Objectives:

1. To understand HPC application.

Theory:

One possible high-performance computing (HPC) application for the artificial intelligence/machine learning (AI/ML) domain is training deep neural networks on large datasets. Deep learning model shave millions of parameters that need to be optimized during training, which can take a long time on a single CPU. HPC system scan accelerate this process by distributing the training work load across multiple processor so even multiple nodes in a cluster.

Here is a brief over view of the steps involved in implementing an HPC application for training deep neural networks:

Choose a deep learning framework: There are many popular deep learning frame works, such as Tensor Flow, Py Torch, and Caffe. Choose one that has good support for distributed training and is compatible with your HPC system.

Prepare the data: The data used for training the neural network should be stored in a format that can be efficiently accessed by the HPC system, such as HDF5 or LMDB. The data should also be split into batches that can be processed in parallel.

Design the neural network architecture: Choose an appropriate neural network architecture that is well-suited to the task at hand. This may involve experimenting with different architectures and hyper parameters to find the best model.

Implement the training algorithm: This involves defining the loss function and optimization method used to train the neural network. Stochastic gradient descent (SGD) with momentum is a popular optimization method for deep learning.

Distribute the training workload: Use a distributed training framework to divide the training workload across multiple processors or nodes in the HPC system. This can be done using data parallelism, model parallelism, or a combination of both. In data parallelism, each processor or node processes a subset of the data and updates the model parameters in parallel. In model parallelism, different processors or nodes are responsible for computing different parts of the model.

Monitor and tune the training process: Monitor the training process to ensure that it is progressing as expected and that there are no performance bottlenecks. Tune the hyperparameters and architecture as needed to improve the accuracy and convergence speed of the model.

Evaluate the trained model: Once the model is trained, evaluate its performance on a validation dataset to assess its accuracy and generalization ability. The trained model can then be used for prediction on new data.